

PHY410/505 midterm, fall 2025

Due date: Thursday, October 23 at 11:59pm

This midterm will test the technical programming aspects covered so far this semester. We've provided some C++ code for a text-based "role playing game": **World of Textcraft**. This is a larger program than those used in assignments so far, with several classes and tests defined in addition to the main executable. While the program does not directly involve much physics, it covers many of the technical topics introduced in the first half of this semester. The same program structure could be used, for example, for an N-body physics simulation, where physical interactions replace the simple hit point calculations.

Instructions

- PHY410: Do problems 1–3. You can do problem 4 for extra credit (it will replace your lowest scoring problem from your homework problems, if higher).
- PHY505: Do all problems, 1–4.

Accept the assignment on GitHub Classroom here: <https://classroom.github.com/classrooms/46534332-compphys-fall2025>. I recommend that you commit and push your progress at regular intervals (e.g. daily), which ensures your progress is backed up. Don't worry, you can commit and/or push as many times as you like; nothing is considered "submitted" until the deadline.

To start, you should read `README.md` to understand the structure of the program. Running `make` from the top-level directory will compile the program; try running some of the test programs under `WorldOfTextCraft/tests`.

Upload your writeup to UBLearn and push your committed code to GitHub Classroom. Your writeup should include documentation of the code you wrote, as well as the solutions to the problems in written form. Do not work with other students.

Docker container

As usual, feel free to use any environment you like, including our Docker container at `ubsuny/compphys:latest`. The usual scripts (`runDocker.sh`, `runDocker_wfix.sh`, and `launchJupyter.sh`) have been copied to this repository.

If you use the Docker container, you might find yourself in need of multiple terminal windows: one for a Jupyter server, another for compiling and running the game. There are a few ways to do this:

- You can connect another terminal to the container using `docker exec...` as follows:
 - First, look up the "contained ID" of your running container using `docker ps` (you might need `sudo`). It will be a random hexadecimal string, for example `dc07315c5228`.
 - (You can directly connect with `docker exec -it dc07315c5228 /bin/bash`. The `-it` calls Docker in interactive+TTY mode, which gives you a shell prompt, and the `/bin/bash` puts you in a bash shell. However, this is not ideal for our container!)
 - However, our container makes use of an "entrypoint" script that runs every time you start the container. So, you should connect to the running container using `docker exec -it dc07315c5228 /usr/local/bin/entrypoint.sh` (or `entrypoint_wfix.sh` if you are using WSL+Ubuntu).
- You can modify the `launchJupyter.sh` script to start the server in the background. Add a `&` character at the end of the `jupyter-lab` command.
- You can open a terminal inside the JupyterLab browser window. Click the "+" as if you were making a new notebook, and the button to start the terminal is down the page.

Problem 1: Example Battle

The configuration files for this problem are located in `WorldOfTextCraft/ExampleBattle`.

Problem 1a

10 points

Compile the program with `make` and run through the example battle. Specifically compile everything with:

```
cd WorldOfTextCraft
make
```

For convenience, you can add the `bin` folder to your `$PATH` variable, which lets you execute `WorldOfTextCraft.exe` from anywhere (otherwise, you'll have to type a full relative or absolute path to the executable, which is fine but a bit painful):

```
cd WorldOfTextCraft
export PATH=$PATH:$(readlink -e bin)
```

To run the example battle:

```
cd WorldOfTextCraft/ExampleBattle
WorldOfTextCraft.exe PlayerCharacters.txt Boss.txt BossScript.txt problem1a.txt
```

For your first turn, use the actions in `ExampleBattle/PlayerScript.txt` (i.e., copy-and-paste them into the interface when prompted). **On subsequent turns, you can just hit “enter” to repeat the previous action** (or you can specify a new action, for example changing the healer's target).

Besides running the game, this will save a battle log to `problem1a.txt`.

Did you win? Copy the outcome to your writeup, and commit `problem1a.txt` to your repository.

Problem 1b

15 points

You will now update the `Rogue.h` and `Rogue.cc` files to enhance your powers (since you are the developer, this is not considered cheating ;)). The class definition is quite short, since it inherits most of its functionality from `Character.h`; `Rogue.h` simply overloads the `attack()`, `defend()`, and `heal()` methods. For `attack()` specifically, it calls a function called `defaultAttack()` which is defined in the base class (i.e., in `Character.h` and `Character.cc`).

For this problem, modify the `Rogue::attack()` method as follows:

- First, copy the contents of `defaultAttack()` into `Rogue::attack()`.
- Every third turn (i.e., for `turn mod 3 == 0`), write a new method to do an extra 20 damage on top of the base attack power.
- Otherwise (i.e., for `turn mod 3 == 1` or `2`), call `defaultAttack()`.

Ensure that `myAttacks_` is up to date!

After updating the `Rogue` class and re-compiling, run the game with the following command:

```
WorldOfTextCraft.exe PlayerCharacters.txt Boss.txt BossScript.txt problem1b.txt
```

Hints: - The `getTurn()` method in the `Character` base class can be used to determine which turn the `Rogue` is on.

Did you win now? Copy the outcome to your writeup, and commit `problem1b.txt` to your repository.

Problem 2: The Lich King battle

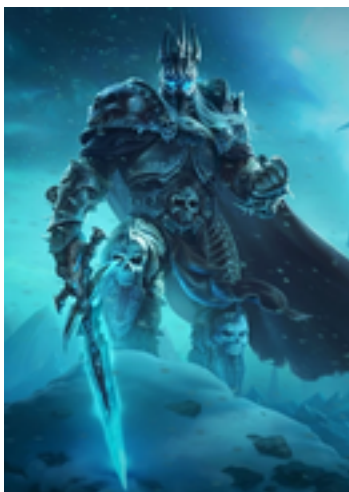


Figure 1: Lich King

For Problem 2, we'll run through a different scenario, the Lich King boss. The configuration files are located at `WorldOfTextCraft/LichKing`. (If you quit your terminal window after the last problem, remember to re-run the `PATH` command so that `WorldOfTextCraft.exe` is on your `PATH`.) Run through this battle with the following commands, including your updated Rogue from Problem 1b:

```
cd LichKing
WorldOfTextCraft.exe PlayerCharacters.txt Arthas.txt ArthasScript.txt problem2.txt
# You can copy-and-paste action commands from LichKing/PlayerActions.txt
```

Commit `problem2.txt` to your repository. Now, the actual problem involves analyzing the battle log file using python. Start the Jupyter server and open the notebook `Problem2.ipynb`. You should do the whole problem in this notebook.

Problem 2a

5 points

The notebook reads in a battle log file using a helper script in `readbattle.py`. The script puts the data into a numpy array for you.

For each character, make a plot visualizing the actions during the battle (`['Attacks', 'Defends', 'Heals', 'DamageReceived', 'HealingReceived']`). Specifically, the x-axis should be the turn number, and the y-axis should be the number corresponding to the action (damage done, damage mitigated, amount healed, etc.), as present in the log file. Label your axes and make a legend.

The first plot for `Attacks` is already done as an example.

Problem 2b

5 points

Using numpy, make a single plot showing the hit points remaining at the end of each turn (y-axis) versus turn number (x-axis) for all characters. Label the axes and draw a legend. You will have to account for both `DamageReceived` and `HealingReceived`. `np.cumsum()` may help.

Problem 2c

5 points

Using the matplotlib hist function, create a single plot showing histograms of the Attacks for each character. Label your axes and make a legend.

Problem 2d

10 points Repeat 2c, but this time plot the Attacks only for turns where the character was healed.

For the submission, please do the following: - Commit the notebook to your repository. - Export the notebook to HTML format with filename `Problem2.html`, and upload to UBLearn.

Problem 3

Enter the physicist

The Warrior, Priest, and Rogue are clearly not getting the job done. It is time for a Physicist to enter the battle.

Problem 3a

15 points

Let's add Isaac Newton to our party. Isaac throws apples at the Boss, which scatter elastically and do damage equal to the kinetic energy transferred to the Boss.

Create a new class, **Physicist**, with the following specifications:

- The Physicist is a damage dealer, so copying **Rogue.h** and **Rogue.cc** is a good starting point.
- Additionally, the Physicist has two more stats, **mana_** and **velocity_**.
 - **mana_**: similar to the Priest and Boss classes. The Physicist should start with 100 mana. This does not need to be configurable.
 - **apple_mass_**: this should be configurable via the configuration files. Besides adding a member variable to the class, you need to override the two **Character::input()** functions to handle one additional argument from the configuration file. (You might want to refer to the **Boss** class definition.)
- In **attack()**, calculate damage as follows. The Physicist throws an apple of mass **apple_mass_** (kg) with velocity $v = \text{attack_power_}$ (m/s). The Boss has a mass of $M = 200$ kg. The apple collides elastically with the Boss and bounces straight backwards (i.e. a 1D collision). The damage is equal to the kinetic energy (joules) imparted to the Boss. Write the code to calculate the kinetic energy imparted to the Boss.
- Besides creating **Physicist.h**, you will need a small addition to **Battle.cc**.

Once you have created the new class, create a new scenario (i.e., the **.txt** configuration files) in a new folder, **LichKing2**. Copy the original **LichKing** configuration files to that folder, and add a new character to your party:

- class = Physicist
- name = Isaac Newton
- attack power = 50
- defense power = 0
- heal power = 0
- apple mass = 1.0

Here are some suggested commands to get started:

```
cd WorldOfTextCraft
mkdir LichKing2
cp LichKing/*txt LichKing2
cd LichKing2
# Add Newton to PlayerCharacters.txt
WorldOfTextCraft.exe PlayerCharacters.txt Arthas.txt ArthasScript.txt problem3.txt
# You can copy-and-paste action commands from LichKing2/PlayerActions.txt
```

Did you (finally) win? Commit **problem3.txt** to your repository.

Problem 3b

10 points

Repeat the data analysis from Problem 2 using the log file from this battle, **LichKing2/problem3.txt**. Start by copying the notebook, **cp Problem2.ipynb Problem3.ipynb**, and remember to update the char-

acter list in the notebook. Once finished, commit the notebook to the repository and upload an HTML export of the notebook to UBLearn.

Problem 4

This problem is mandatory for PHY505 students. PHY410 students can also complete the problem for extra credit.

In this problem, we will add randomness to the game. C++ has quite a few utilities for generating (pseudo)random numbers (see <https://en.cppreference.com/w/cpp/numeric/random.html>). In this problem, we will use the Poisson distribution to randomly vary the program.

Pseudorandom number generators are often used in programming to create a sequence of numbers that appear random, but are generated in a deterministic way. For most purposes, they can be used like real random numbers, but since the sequences are deterministic, our programs' results are fully reproducible, which is usually desired in programming (for unit tests, for example). Pseudorandom number generators are usually initialized with a “seed,” which is just an integer used in some way to start the sequence. For a given seed, the generator always yields the same sequence; the seed can be varied to generate different sequences.

We will use the `std::default_random_engine` class for our pseudorandom number generator. The random seed is obtained from `std::random_device`, which generates a true random number. Finally, in `std::poisson_distribution<>`, the pseudorandom numbers are used to sample the Poisson distribution.

Problem 4a

20 points

Add a random number generator to the Rogue class's attack. Following closely this example, do the following steps to introduce random variations in the characters' actions:

- In the Character class, add an instance of `std::default_random_engine` as a class member variable.
- In the constructor `Character::Character()`, create the generator and give it a random seed using `std::random_device`, following the example.
- In the Character class, add an instance of `std::poisson_distribution` as a class member variable.
- **Important:** you cannot initialize the `std::poisson_distribution` in the constructor, because `attack_power_` has not yet been set! Instead, initialize `poisson_` in the function `Rogue::post_input() override {...}`. This is a “hook” function declared in the base class that you can override in the derived class to run some code directly after `input()`.
 - We have to use this hook because `attack_power_` is not set until `input()` is run.
- Finally, change `Rogue::attack()` to use `poisson_` to generate a random attack value, instead of using `attack_power_` directly. Make sure to preserve the modifications from Problem 1b.
- Compile with `make`.

Once you've finished adding random numbers to the Rogue class, create a new scenario in folder `LichKing3`. Copy the configuration files from Problem 3 (`LichKing2`). (If you didn't finish Problem 3, you can copy from `LichKing` instead.) Run the scenario without any further modifications, e.g.,

```
cd WorldOfTextCraft
mkdir LichKing3
cp LichKing2/*.txt LichKing3
cd LichKing3
WorldOfTextCraft.exe PlayerCharacters.txt Arthas.txt ArthasScript.txt problem4.txt
# You can copy-and-paste action commands from LichKing2/PlayerActions.txt
#
```

You should now see random values for the Rogue's damage. Commit `problem4.txt` to your repository.

Problem 4b

5 points

Repeat Problem 2c: start by copying the notebook, `cp Problem2.ipynb Problem4.ipynb`, and change the input to `LingKing3/problem4.txt`. Remember to update the character list. Using the `matplotlib.hist` function, create a single plot showing histograms of the Attacks for each character. Label your axes and make a legend.

You should now see a Poisson distribution for the Rogue's damage.

Upload the notebook to your repository, and upload an HTML export of the notebook to UBLearn.